



Database selection criteria 10m

Use database selection criteria when choosing a database. Common criteria that web developers will consider when determining if a database is suitable are:

- cost
- stability and reliability of the service
- performance
- security
- scalability
- ease of using the database
- **active support** from vendors and the community.

Cost

It is best to consider cost first as you will find it will be a deciding factor on whether to move forward with a specific database. When thinking about cost, you need to consider whether you'll use an open-source or licensed database because this will affect the cost.

You need to investigate:

- purchase cost
- maintenance and operation costs
- subscription fees, charges for use and any hidden costs
- troubleshooting cost and support
- upgrade and migration costs
- storage and network requirement costs
- expert or specialist support costs.

[Skip to main content](#)

Performance

A database with good performance will create a better user experience when they access your application, which is why you need to consider performance. You need to think about:

- latency
- queries per second
- support for more advanced hierarchical storage features.

Security

One of the most important considerations is security because it is vital your database is secure. Because each web application is different, the security techniques that need to be adopted may differ. Think about the following points when selecting a database:

- What database measures can help prevent SQL injections and information leakage?
- Is there an audit feature that will fulfil the auditing requirements?
- Can sensitive data be encrypted?
- Can the technology control access and permissions for different users and groups?

Usability and scalability

You should also consider whether your database service is stable, can scale and is usable by users and web developers. Think about whether:

- there is support for multiple data replicas
- there is an availability of services you'll need, such as special database features
- the architecture supports multiple writes and multi-active architecture
- it has a reliable system to monitor and alert the flow of data
- there is support for backups and restoring data
- there is vendor support and a community
- there is ease of performance tuning
- it's easy to scale horizontally and vertically.

[Skip to main content](#)

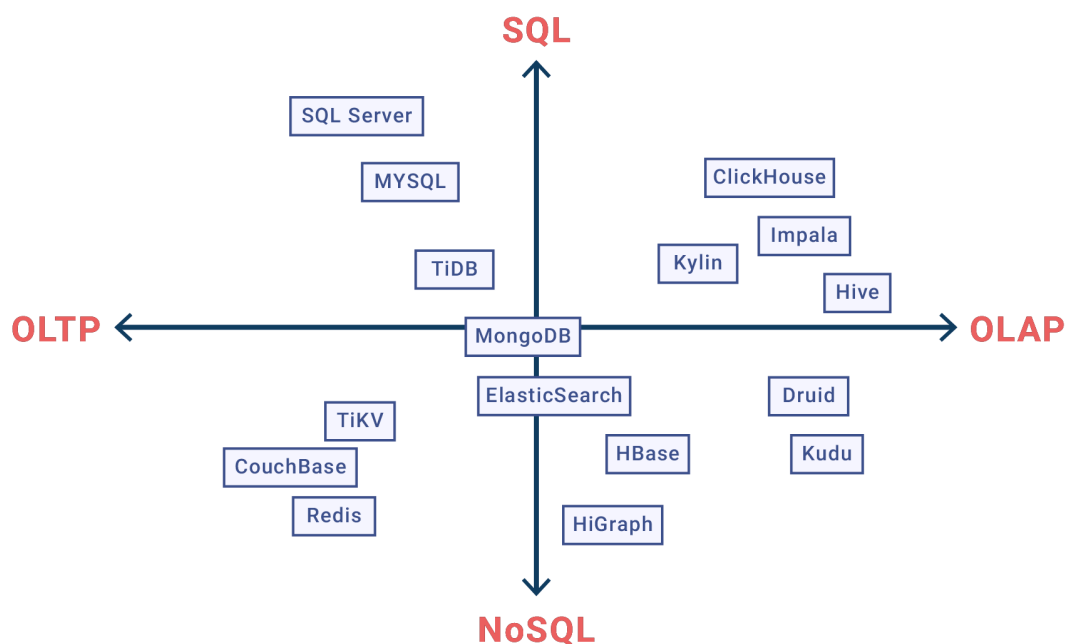
Categorising databases

Since there are so many databases, it can be hard for web developers to know which database is the most suitable for their web application. The matrix below is helpful for deciding what type of database to use depending on the web application's requirements and the database interface you need.

- The X-axis represents the requirements that a web application may have, which are either online transactional processing (OLTP) or online analytical processing (OLAP).
- The Y-axis represents the database interface that the web application will use, which is either SQL or NoSQL.



Database categories



[Skip to main content](#)



As you can see, this matrix shows that:

- Databases in the upper left quadrant support OLTP workloads and the SQL language. For example, MySQL supports various transaction isolation levels, high queries per second (QPS), and low latency—making it ideal for storing transaction information and critical data in such scenarios.
- Databases in the lower left quadrant typically support OLTP workloads and NoSQL query languages. These NoSQL databases are designed to optimize performance for specialised applications and requirements. They are often chosen for their simple or schema-less structures, high throughput, and low latency. As such, they are well-suited for use as caches or key-value (KV) stores.
- Databases on the right are all OLAP big data analytical systems. Generally, they support the SQL language and don't support transactions. They have good scalability and long response latency. Although extra machines can be added to store more data, the response delay is longer.
- Databases around where the two axes intersect are neutral, and we call them HTAP (hybrid transaction/analytical processing) databases. When the data capacity is small, then the performance will be good. Likewise, when the data size is large or the queries are complex, their performance will be poor.

Generally, to meet different application needs, it is ideal to use different storage engines and query engines.



The next section will show you how to connect a database to the client side.



[Skip to main content](#)

Thoughts on this page?



Proprietary versus open source databases

How would you pick between the two from a business perspective? What would be some deciding factors?

B *I* ☰ 🔗

Write your comment

📁 Add file

Cancel

Send

Sort by:



Jorge Andres Guerrero Giraldo 15 Jan 2026, 9:43 AM

From a business perspective, the choice depends on **cost, risk and long term strategy**.

Open source databases are ideal when budget is limited, flexibility is important, and there is strong community support. Proprietary databases are often chosen when guaranteed vendor support, compliance requirements and enterprise level reliability are critical.

